TRƯỜNG ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Group 4

1. Đoàn Đức Anh - 202417091
2. Bùi Đức Sơn - 202417193
3. Phạm Lê Công - 202417102
4. Phạm Đức Gia Bảo - 202417101
5. Nguyễn Văn Tùng Cương - 202417103

Tetris Game

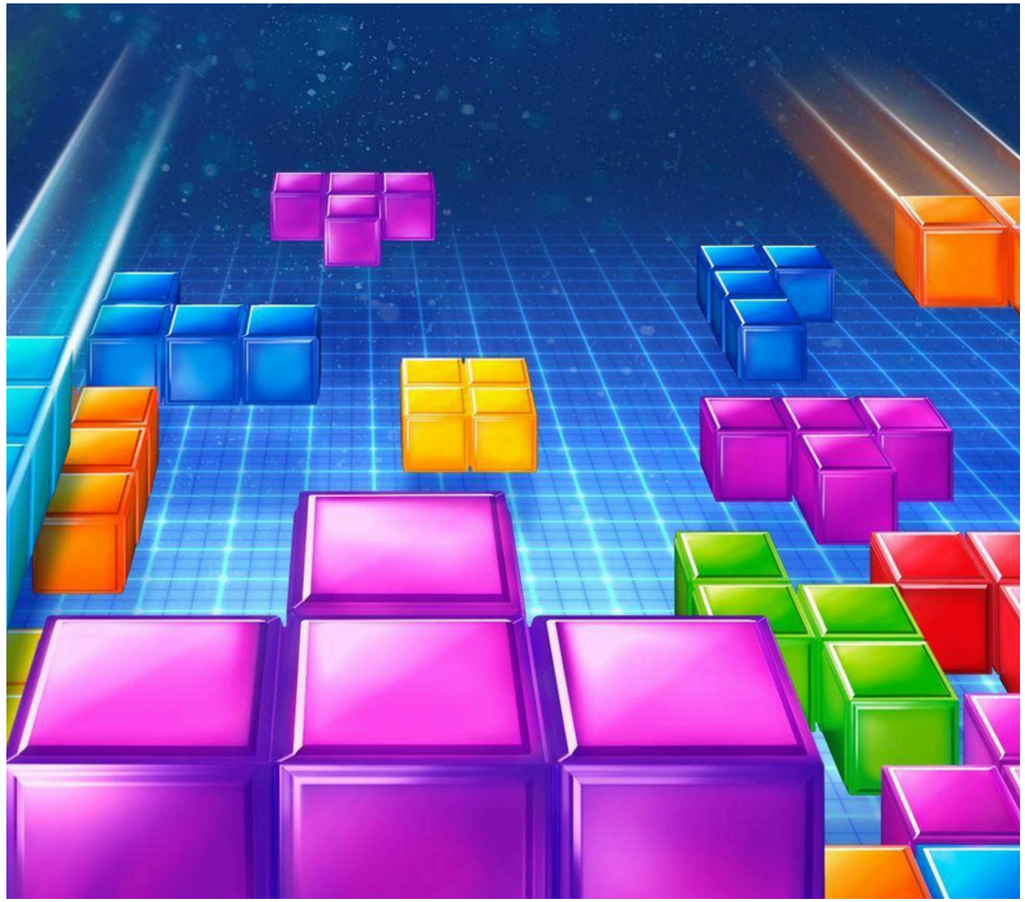
ONE LOVE. ONE FUTURE.

1. Member and assignment

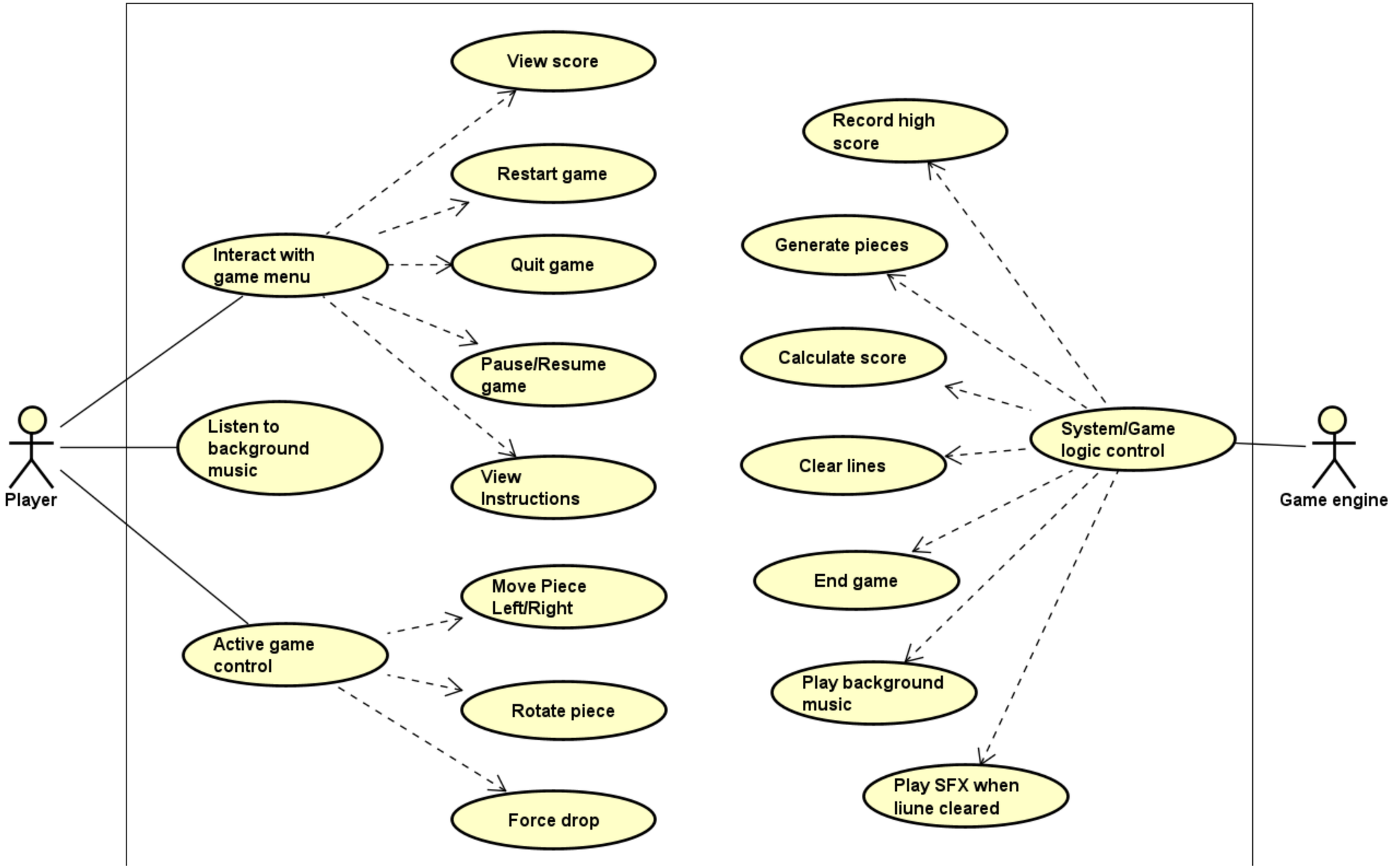
Group members	Student ID	Class/methods assigned	Percentage (%)
Đoàn Đức Anh	202417091	Classes: Cell, Board, Tetromino, StraightShape, SquareShape, IShape, SShape, ZShape, TShape,JShape, Main	20
Bùi Đức Sơn	202417193	Classes: MenuPanel, HelpPanel, SoundController, GamePanel(modify: bgmToggleButton), DifficultyPanel (modify: difficultyComboBox, paintComponent)	20
Phạm Lê Công	202417102	Classes: MainFrame, GamePanel(modify: drawNextPieces, drawScore), GameEngine(modify: spawnNewPiece, createNextPiece, addPiecetoQueue, getNewPiece, getUpcomingPieces	20
Phạm Đức Gia Bảo	202417101	Classes: GamePanel, DifficultyPanel, Constants, GameEngine(modify: scoreMultiplier, calculateScore, setDifficultyDelay), MainFrame(modify: constructor), HelpPanel(modify: constructor)	20
Nguyễn Văn Tùng Cương	202417103	Classes: GameEngine, GamePanel(modify: setEngineAndRouting, setupKeyBindings, paintComponent, drawSingleBlock, drawNextPieces, drawGrid, drawLockedBlocks, drawActivePiece, drawShadowPiece, drawScore, setActivePiece, refreshBoard)	20

2. Problem statement

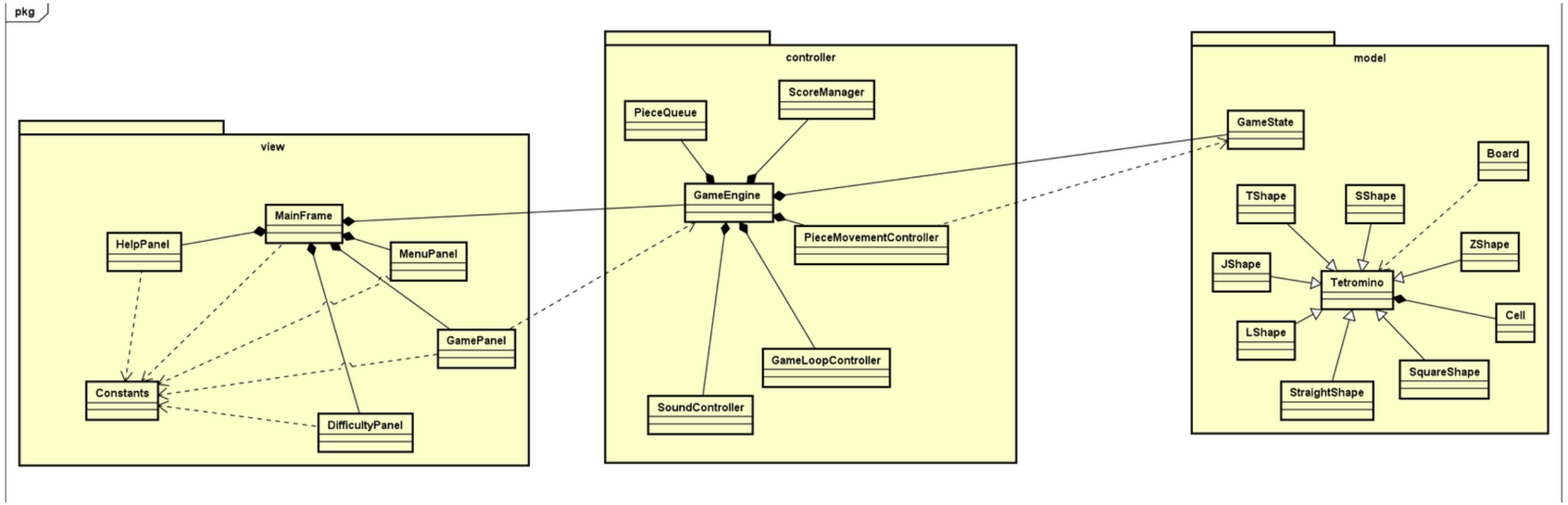
Tetris is a classic puzzle game where players arrange falling blocks to clear horizontal lines. This project applies object-oriented programming (OOP) to build a structured, maintainable, and extensible system.



3. Use case diagram



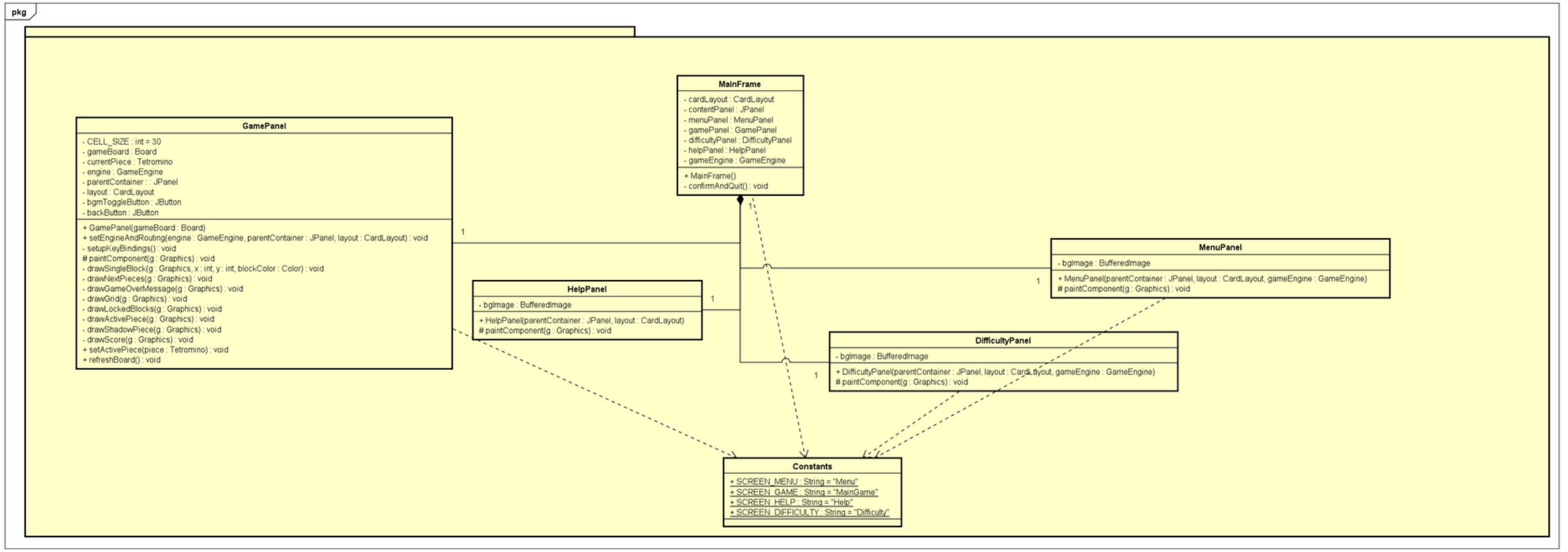
4. General class diagram



MVC architecture

- The system follows the Model–View–Controller (MVC) architectural pattern.
- The application is divided into three core packages: model, view, and controller.
- The Controller (GameEngine) mediates between user actions and game state updates.
- The View captures user input and renders the current game state without embedding game logic.
- The Model encapsulates all game data and rules, remaining independent of UI and control flow.
- This structure ensures low coupling, high cohesion, and easy extensibility

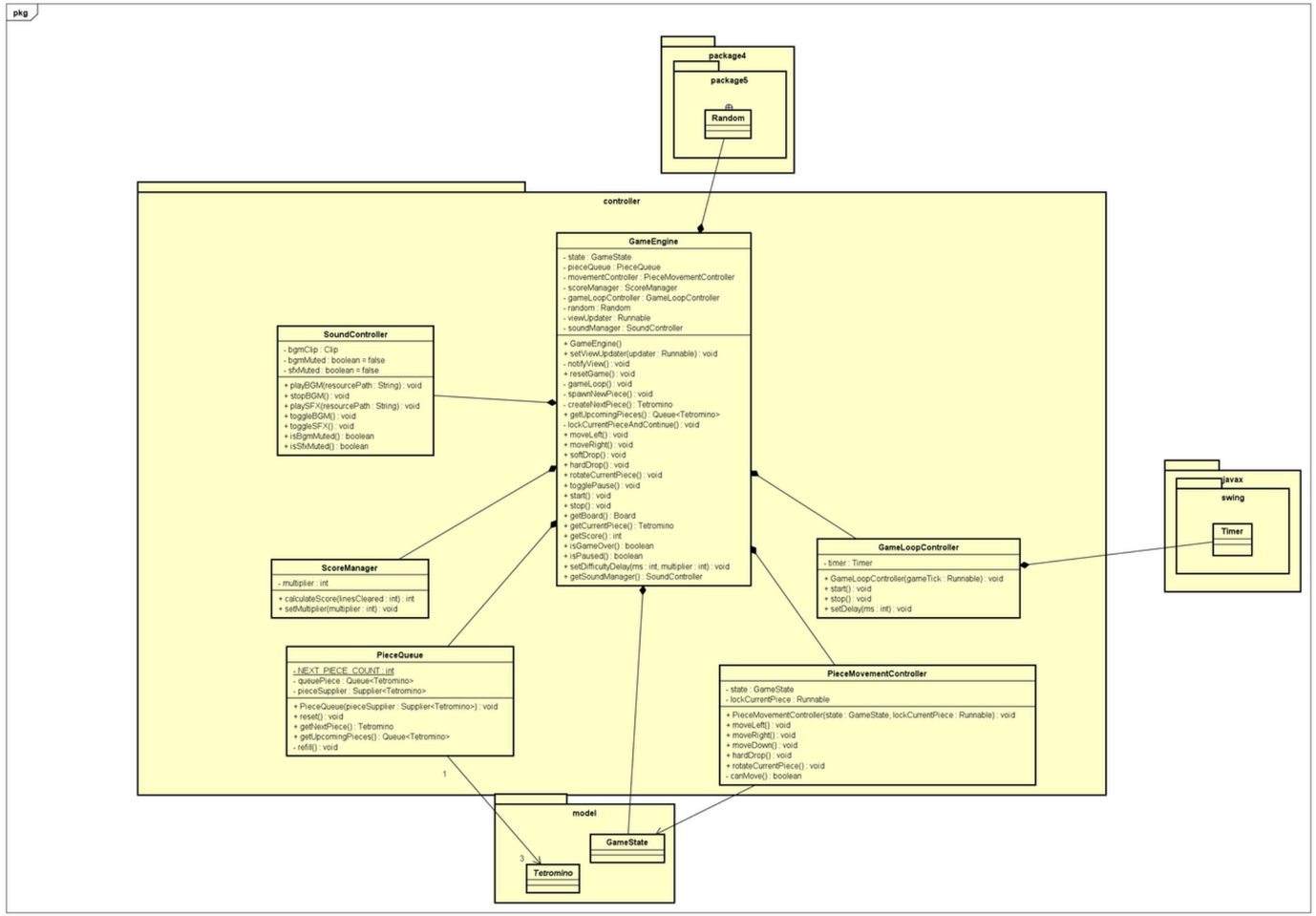
5. View package class diagram



- Implemented using Java Swing and AWT.
- MainFrame serves as the main window and manages screen navigation using CardLayout.
- UI screens (MenuPanel, GamePanel, HelpPanel, DifficultyPanel) are owned by MainFrame.
- The View layer is purely passive and never modifies game state directly

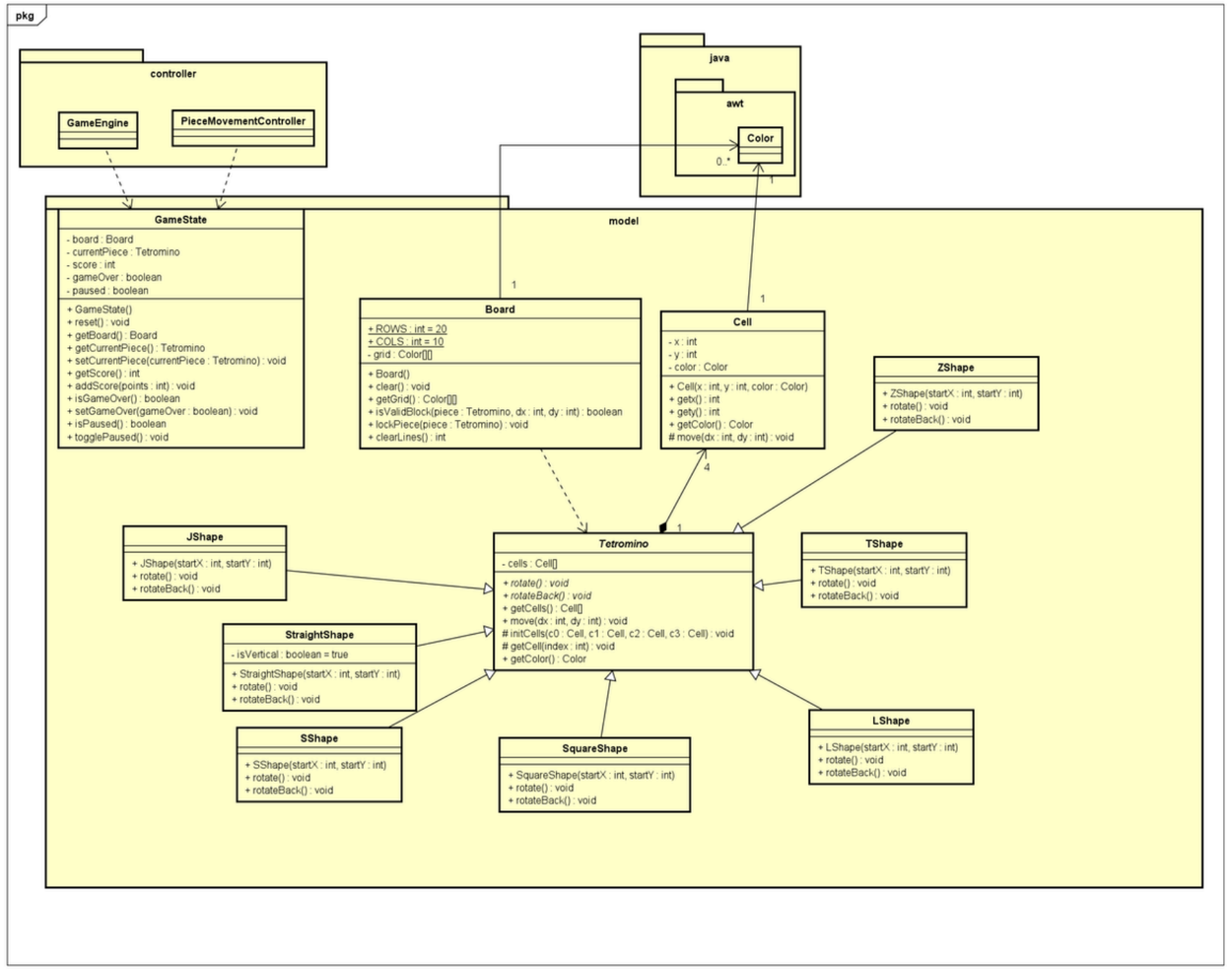
6. Controller package class diagram

- The GameEngine acts as the central controller of the application.
- It manages the game loop, piece movement, rotation, scoring, and game state transitions.
- A Swing Timer drives the falling logic based on selected difficulty.
- The controller validates all moves against the model before applying changes.
- View updates are triggered through a callback mechanism, preserving MVC separation.
- SoundController handles background music and sound effects independently.



7. Model Package class diagram

- The model package encapsulates all core game data and logic.
- Board represents the 20×10 playing grid and manages collisions and line clearing.
- Tetromino is an abstract base class defining movement and rotation behavior.
- Each Tetromino is composed of four Cell objects, representing atomic blocks.
- Concrete Tetromino subclasses implement shape-specific rotation logic.
- The Model layer contains no UI or input dependencies, ensuring data integrity.



8. OOP technique used

ABSTRACTION

Example of usage:

- Tetromino class is decleared as an abstract class with abstract methods: rotate() and rotateBack() which are later used or Overriding.

```
public abstract class Tetromino {
    protected Cell[] cells = new Cell[4];
    protected Color pieceColor;

    public abstract void rotate();
    public abstract void rotateBack();

    public Cell[] getCells() { return cells; }

    public void move(int dx, int dy) {
        for (Cell cell: cells){
            cell.setx(cell.getx() + dx);
            cell.sety(cell.gety() + dy);
        }
    }
}
```

8. OOP technique used

ENCAPSULATION

Example of usage:

- Private attributes inside Cell class (x, y, color) are only accessible and modifiable by getters and setters (getx(), gety(), getcolor(), ...).

```java
public class Cell {
    private int x;
    private int y;
    private Color color;

    public Cell(int x, int y, Color color) {
        this.x = x;
        this.y = y;
        this.color = color;
    }

    public int getx() {
        return x;
    }
    public int gety() {
        return y;
    }

    protected void move(int dx, int dy) {
        this.x += dx;
        this.y += dy;
    }

    public Color getColor() {
        return color;
    }
}
```

8. OOP technique used

INHERITANCE

Example of usage:

- JShape class is made by inheriting from Tetromino class and MenuPanel is made by inheriting from JPanel class.

```
public class JShape extends Tetromino {
    public JShape(int startX, int startY) {
        Color color = Color.CYAN;

        initCells(
            new Cell( x: startX + 1, startY, color),
            new Cell( x: startX + 1,  y: startY + 1, color),
            new Cell( x: startX + 1,  y: startY + 2, color),
            new Cell(startX,  y: startY + 2, color)
        );
    }

    @Override
    public void rotate() {...}

    @Override
    public void rotateBack() {...}
}
```

```
public class MenuPanel extends JPanel {
    private BufferedImage bgImage;

    public MenuPanel(JPanel parentContainer, CardLayout layout, GameEngine gameEngine) {...}

    @Override
    protected void paintComponent(Graphics g) {...}
}
```

8. OOP technique used

POLYMORPHISM

Example of usage:

- currentPiece attributes in GameEngine class is decleared as abstract type Tetromino which holds any one of 7 different shape objects at runtime.
- createNextPiece in GameEngine is a method that can return any of the 7 different shape objects.

```
private Tetromino createNextPiece() {
    int shapeType = random.nextInt( bound: 7);
    if (shapeType == 0) {
        return new StraightShape( startX: Board.COLS / 2,  startY: -1);
    } else if (shapeType == 1) {
        return new SquareShape( startX: Board.COLS / 2,  startY: -1);
    } else if (shapeType == 2) {
        return new LShape( startX: Board.COLS / 2,  startY: -1);
    } else if (shapeType == 3) {
        return new JShape( startX: Board.COLS / 2,  startY: -1);
    } else if (shapeType == 4) {
        return new TShape( startX: Board.COLS / 2,  startY: -1);
    } else if (shapeType == 5) {
        return new SShape( startX: Board.COLS / 2,  startY: -1);
    } else {
        return new ZShape( startX: Board.COLS / 2,  startY: -1);
    }
}
```

8. OOP technique used

COMPOSITION

Example of usage:

- GameEngine class is built entirely by composing other objects as attributes. It has a "has-a" relationship with every component it needs.

```
public class GameEngine {
    private GameState state;
    private PieceQueue pieceQueue;
    private PieceMovementController movementController;
    private ScoreManager scoreManager;
    private GameLoopController gameLoopController;
    private final Random random;
    private Runnable viewUpdater;
    private SoundController soundManager;

    public GameEngine() {
        random = new Random();
        state = new GameState();
        scoreManager = new ScoreManager();
        pieceQueue = new PieceQueue(this::createNextPiece);
        movementController = new PieceMovementController(state, this::lockCurrentPieceAndContin
        gameLoopController = new GameLoopController(this::gameLoop);
        soundManager = new SoundController();
    }
```

8. OOP technique used

Aggregation

- Has-A Relationship: GamePanel holds a reference to a Board instance (gameBoard).
- Independent Lifecycles: The Board object is created externally and passed into the GamePanel constructor.

```
public class GamePanel extends JPanel {
    private final int CELL_SIZE = 30;
    private Board gameBoard;
    private Tetromino currentPiece;
    private controller.GameEngine engine;
    private JPanel parentContainer;
    private CardLayout layout;
    private JButton bgmToggleButton;
    private JButton backButton;

    public GamePanel(Board gameBoard) {
        this.gameBoard = gameBoard;
        this.setBackground(Color.BLACK);
```

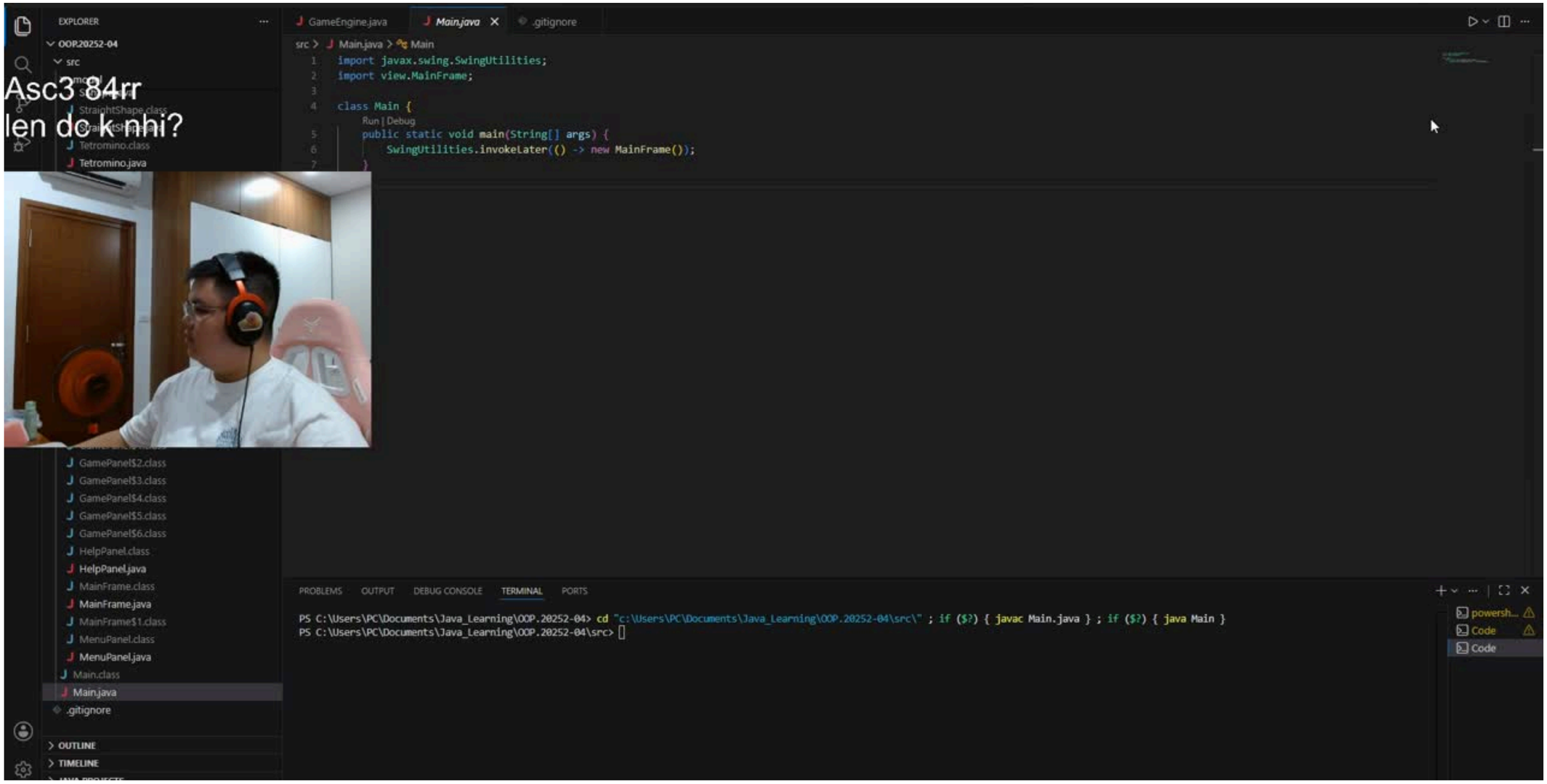
8. OOP technique used

Dependency

- The Board class depends external class to function properly
- Tetromino: Used as a parameter type in the isValidBlock method to check piece placement.

```java
public class Board {
    public static final int ROWS = 20;
    public static final int COLS = 10;

    private final Color[][] grid;

    public Board() { grid = new Color[ROWS][COLS]; }

    public void clear() {...}

    public Color[][] getGrid() {...}

    public boolean isValidBlock(Tetromino piece, int dx, int dy){
        for (Cell cell: piece.getCells()) {
            int newX = cell.getx() + dx;
            int newY = cell.gety() + dy;

            if (newX < 0 || newX >= COLS || newY >= ROWS) {
                return false;
            }

            if (newY >= 0 && grid[newY][newX] != null) {
                return false;
            }
        }
```

9. Demo video


Asc3 84rr
len do k nhi?
EXPLORER
OOP.20252-04
src
Tetromino.class
Tetromino.java
GameEngine.java
Main.java
.gitignore
src > Main.java > Main
import javax.swing.SwingUtilities;
import view.MainFrame;
class Main {
Run | Debug
public static void main(String[] args) {
SwingUtilities.invokeLater(() -> new MainFrame());
}
GamePanel$2.class
GamePanel$3.class
GamePanel$4.class
GamePanel$5.class
GamePanel$6.class
HelpPanel.class
HelpPanel.java
MainFrame.class
MainFrame.java
MainFrame$1.class
MenuPanel.class
MenuPanel.java
Main.class
Main.java
.gitignore
OUTLINE
TIMELINE
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\PC\Documents\Java_Learning\OOP.20252-04> cd "c:\Users\PC\Documents\Java_Learning\OOP.20252-04\src\" ; if ($?) { javac Main.java } ; if ($?) { java Main }
PS C:\Users\PC\Documents\Java_Learning\OOP.20252-04\src>
powersh...
Code
Code

HUST

hust.edu.vn fb.com/dhbkhn

THANK YOU !